



Piper SDK 逐行精读：piper_ctrl_go_zero.py

机械臂关节 + 夹爪归零（回零）脚本 · 基于本仓库 [piper_tools/piper_ctrl_go_zero.py](#) 与 piper_sdk V2 真实源码整理 ·

2026-06-10

这份脚本是什么？ 它是最小可运行的 Piper SDK 控制示例 —— 连接一只机械臂，把 6 个关节和夹爪全部送回 0 位。麻雀虽小，却完整覆盖了「连接 → 使能 → 切模式 → 下发关节 → 下发夹爪」这条 SDK 控制主线。吃透它，后面所有 ROS2 节点里的臂控制代码都能看懂。

一、完整源码

```
1  #!/usr/bin/env python3
2  # -*-coding:utf8 -*-
3  # 注意 demo 无法直接运行，需要 pip 安装 sdk 后才能运行
4  import time
5  from piper_sdk import *
6
7  if __name__ == "__main__":
8      piper = C_PiperInterface_V2("can_right_slave")
9      piper.ConnectPort()
10     while( not piper.EnablePiper()):
11         time.sleep(0.01)
12     factor = 57295.7795 #1000*180/3.1415926
13     position = [0,0,0,0,0,0,0]
14
15     joint_0 = round(position[0]*factor)
16     joint_1 = round(position[1]*factor)
17     joint_2 = round(position[2]*factor)
18     joint_3 = round(position[3]*factor)
19     joint_4 = round(position[4]*factor)
20     joint_5 = round(position[5]*factor)
21     joint_6 = round(position[6]*1000*1000)
22     piper.ModeCtrl(0x01, 0x01, 30, 0x00)
23     piper.JointCtrl(joint_0, joint_1, joint_2, joint_3, joint_4, joint_5)
24     piper.GripperCtrl(abs(joint_6), 1000, 0x01, 0)
```

二、逐行讲解

① 导入与入口 (L1-L7)

行	代码	含义
L4	<code>import time</code>	仅用于使能轮询时的 <code>sleep(0.01)</code> 。
L5	<code>from piper_sdk import *</code>	导入 Agilex 官方 Python SDK。核心类 <code>C_PiperInterface_V2</code> 由此而来。（生产代码建议显式导入而非 <code>*</code> ）
L3 注释		提醒：demo 需先 <code>pip install piper_sdk</code> 才能跑。本机 SDK 在 <code>/data1/miniconda3/.../piper_sdk/</code> 。

② 连接机械臂 (L8-L9)

```
piper = C_PiperInterface_V2("can_right_slave")  
piper.ConnectPort()
```

- **L8**：实例化 V2 接口，参数是 **CAN 符号名** `"can_right_slave"` —— 这正是第 1 层 CAN 标定时由 `calibrate_can_mapping.py` 写进 `config/pipers.yml` 的那个符号名。换臂只需改这里（如 `can_left_slave`）。
- **L9**：`ConnectPort()` 打开 CAN socket 并启动 SDK 的后台收发线程，开始监听 0x2xx 反馈帧。**必须先连接，后续任何读写才有效。**

为什么是 V2? `C_PiperInterface_V2` 相比 V1 增加了 `judge_flag` 等参数、更完整的限位保护。本仓库的 ROS2 节点（`arm_reader_node.py` 等）也统一用 V2。

③ 使能握手 (L10-L11) ★ 含隐藏陷阱

```
while( not piper.EnablePiper()):  
    time.sleep(0.01)
```

使能 = 给 6 个关节电机 + 夹爪上电。不使能时电机是「软」的，下发任何 `JointCtrl` 都不动。看 SDK 里 `EnablePiper()` 的真实实现：

```
def EnablePiper(self) -> bool:
```

```
enable_list = self.GetArmEnableStatus() # ← 先读“当前”使能状态
self.EnableArm(7) # ← 再发使能指令(7 = 6关节+夹爪 bitmask)
return all(enable_list) # ← 返回的是“发指令之前”的状态
```

隐藏陷阱：返回值是「上一拍」的状态。

`EnablePiper()` 先采样 `enable_list`，再发使能指令，返回的却是**采样时（即指令生效前）**的状态。

所以第一次调用几乎必然返回 `False`。

这正是为什么要用 `while` 循环：每 10ms 轮询一次，直到某一拍读到「全部已使能」才跳出 —— 本

质是等待电机真正上电完成的握手，而不是一次性开关。**不要**把它改成 `if`。

④ 单位换算因子（L12）

```
factor = 57295.7795 # = 1000 * 180 / π
```

SDK 的 `JointCtrl` 要求关节角单位是 **0.001°（毫度）**，而我们习惯用**弧度**。这个因子一步到位完成 **弧度 → 毫度**：

- `180/π`：弧度 → 度
- `×1000`：度 → 0.001°（毫度）

例：1 rad $\times$ 57295.7795 $\approx$ 57296 $\rightarrow$ 即 57.296° $\rightarrow$ 即 57296 个「0.001°」单位。

⑤ 目标位置与换算（L13–L21）

```
position = [0,0,0,0,0,0,0] # 6 关节 + 1 夹爪, 全 0 = 回零
joint_0..joint_5 = round(position[i] * factor) # 弧度 → 毫度
joint_6 = round(position[6] * 1000 * 1000) # 米 → 0.001mm
```

变量	对应	换算	SDK 目标单位
----	----	----	----------

`joint_0..5` 关节 1-6 $\times$ `factor` (rad $\rightarrow$ mdeg) 0.001°

`joint_6` 夹爪开口 $\times$ `1e6` (m $\rightarrow$ 0.001mm) 0.001mm

夹爪换算 $\times 1000 \times 1000$ ：米 $\rightarrow$ 毫米 ($\times 1000$) $\rightarrow$ 0.001mm (再 $\times 1000$)。例：0.07 m 开口 $\rightarrow$

70000 单位（即 70 mm）。此处全 0 表示夹爪闭合到零位。

⑥ 切换控制模式（L22）

```
piper.ModeCtrl(0x01, 0x01, 30, 0x00)
```

下发关节指令前**必须**先把臂切到「CAN 指令控制 + 关节运动」模式，否则臂会忽略 JointCtrl。四个参数（来自 SDK 真实签名）：

位置	参数	本例值	含义
1	ctrl_mode	0x01	CAN 指令控制模式（0x00=待机）
2	move_mode	0x01	MOVE J（关节插补）；其它：0x00 P / 0x02 L 直线 / 0x03 C 圆弧
3	move_spd_rate_ctrl	30	速度百分比 0-100。 30 = 慢速 ，回零时安全
4	is_mit_mode	0x00	位置-速度模式（非 MIT 力控）

底层：ModeCtrl 实际转发 MotionCtrl_2，CAN ID = 0x151。

⑦ 下发关节角（L23）

```
piper.JointCtrl(joint_0, joint_1, joint_2, joint_3, joint_4, joint_5)
```

把 6 个关节目标（毫度）发给臂，CAN ID = 0x155/0x156/0x157（每帧带 2 个关节）。SDK 内部会对每个关节做 __CalJointSDKLimit 限位裁剪，关节硬限位如下：

关节	限位 (°)	关节	限位 (°)
J1	[-150, 150]	J4	[-100, 100]
J2	[0, 180]	J5	[-70, 70]
J3	[-170, 0]	J6	[-120, 120]

全 0 在所有关节限位区间内（注意 J2 范围 [0,180]、J3 范围 [-170,0]，0 是它们的端点），所以「全零」是合法且安全的回零目标。

⑧ 下发夹爪（L24）

```
piper.GripperCtrl(abs(joint_6), 1000, 0x01, 0)
```

位置	参数	本例值	含义
1	<code>gripper_angle</code>	<code>abs(joint_6)</code>	开口，0.001mm。 <code>abs()</code> 保证非负（开口不能为负）
2	<code>gripper_effort</code>	<code>1000</code>	力矩，0.001 N · m → 1 N · m （范围 0–5000）
3	<code>gripper_code</code>	<code>0x01</code>	使能夹爪（0x00 失能 / 0x03 使能并清错）
4	<code>set_zero</code>	<code>0</code>	不设零点（0xAE 才会把当前位置设为 0）

CAN ID = 0x159。注意 `GripperCtrl` 发送后会检查 CAN 返回，失败会 log error。

三、一张图看懂控制主线

```
C_PiperInterface_V2("can_right_slave")  实例化（绑定 CAN 符号名）
ConnectPort()                            打开 CAN，启后台收发线程
while not EnablePiper(): sleep           握手：轮询直到电机全部上电
ModeCtrl(0x01,0x01,30,0x00)             切到「CAN 控制 + MOVE_J + 30%速」
JointCtrl(j0..j5)                       下发 6 关节目标（毫度，自动限位）
GripperCtrl(open, effort, 0x01, 0)      下发夹爪（开口+力矩+使能）
▼ 真臂运动到全零位
```

四、动手练习

练习 1 · 原样运行

```
python piper_tools/piper_ctrl_go_zero.py
```

观察右 slave 臂缓慢回到全零位。（确保已 `bash piper_tools/setup_can.sh` 激活 CAN）

练习 2 · 改目标位姿

把 `position` 改成 `[0, 1.0, -1.0, 0, 0, 0, 0.05]`（J2=1rad、J3=-1rad、夹爪开 50mm），观察

单臂摆姿 + 夹爪张开。先确认每个值在限位内。

练习 3 • 换臂

把 L8 的 `"can_right_slave"` 改成 `"can_left_slave"`，控制左臂。对照 `config/pipers.yml` 里的符号名。

安全提醒：① 首次务必用低速 (`move_spd_rate_ctrl ≤ 30`)；② 改 `position` 前核对关节限位表；③ 真臂周围清空、手放急停附近；④ 这是 **slave**（执行）臂，不要对 **master**（示教）臂跑此脚本。

五、延伸：和上层代码的关系

- **遥操作：**`arm_teleop_node.py` 把 master 臂的关节读数实时喂给 `JointCtrl`，原理与本脚本一致，只是目标来自另一只臂而非常量。
- **自主推理：**`policy_inference_node.py` 把 VLA 模型输出的动作经 `JointCtrl/GripperCtrl` 下发，同一套 API。
- **预对齐：**`go_to_pose.py` 是本脚本的「平滑版」—— 从当前位姿插值到目标，而非一步到位，用于 VLA 推理前回到 demo 起始位。

配套学习手册见 `docs/piper_arm_learning_guide.md` 第 2 层。本文 API 签名摘自 `piper_sdk V2` `piper_interface_v2.py` 实际源码。